🎓 CIÊNCIA DA COMPUTAÇÃO

Grafos e Probabilidade

Marco 5 — Apresentação Final:

Round Trip II em Grafos Dirigidos



EQUIPE DE DESENVOLVIMENTO
Yan Felipe & Antonio Eduardo

PROBLEMA ATRIBUÍDO
CSES 1678 • Round Trip II

STATUS FINAL
ACCEPTED

O Problema Byteland e a Instância de Exemplo

Objetivo do Problema

Planejar uma viagem de ida e volta que inicie em uma cidade, visite cidades intermediárias distintas e retorne à origem utilizando apenas **voos só de ida**.

Grafo da Instância (N = 4, M = 5)

- **Vértices:** V = {1, 2, 3, 4} (Cidades)
- **Arestas dirigidas:** (1 → 3), (2 → 1), (2 → 4), (3 → 2), (3 → 4)
- **Solução Válida:** Rota 1 → 3 → 2 → 1 (k = 4)



⚠ Aresta de Retorno Fechando o Ciclo: (2 → 1)

Definição Teórica: O que é um Ciclo Dirigido?

Propriedade Estrutural

Um **ciclo simples dirigido** é um caminho fechado **v1 → v2 → ... → vk → v1** no qual todos os vértices **vi** intermediários são estritamente distintos e cada par consecutivamente ligado é um arco do grafo.

> **Aresta de Retorno (Back-Edge):**
> Um ciclo existe se, e somente se, a DFS encontra um arco apontando para um vértice que **já está presente na pilha de recursão atual**.

Controle por Pilha de Recursão

Acompanhamos o estado da busca com estruturas auxiliares paralelas (visitado e na_pilha):

- **visitado[u] = False**
  Vértice totalmente inexplorado no grafo.
- **na_pilha[u] = True**
  Vértice ATIVO no caminho atual da DFS (Pilha de Recursão).
- **na_pilha[u] = False (após desempilhar)**
  Processamento das ramificações do nó finalizado.

Critério Algorítmico: Detecção da Aresta de Retorno

Um ciclo é identificado no instante em que a DFS tenta navegar para um vizinho cujo estado atual é **na_pilha[v] == True**.

Passo a Passo da Execução:

1. **DFS(1):** Marca na_pilha[1] = True. Vai para o nó 3.
2. **DFS(3):** Marca na_pilha[3] = True. Vai para o nó 2.
3. **DFS(2):** Marca na_pilha[2] = True. Analisa vizinho 1.
4. **DETECÇÃO: Como na_pilha[1] == True, a aresta (2 → 1) é uma aresta de retorno!**

Reconstrução via Vetor pai[]:
Retrocede-se de 2 até 1 usando os antecessores: 2 → pai[2](3) → pai[3](1), fechando a rota 1 → 3 → 2 → 1.

Tabela de Estado das Estruturas Auxiliares

Vértice (u)	Estado (`na_pilha`)	Pai (`pai[u]`)
1	NA PILHA (True)	0
3	NA PILHA (True)	1
2	NA PILHA (True)	3
4	FORA DA PILHA (False)	0

Identificação do Ciclo na Classe DirectedCycle

```
class DirectedCycle:
    def _dfs(self, u):
        self.na_pilha[u] = True   # Empilha o vértice u
        self.visitado[u] = True

        for v in self.adj[u]:
            if not self.visitado[v]:
                self.pai[v] = u
                if self._dfs(v):
                    return True
            elif self.na_pilha[v]:
                # DETECÇÃO: Vizinho v já está NA PILHA!
                self.inicio_ciclo = v
                self.fim_ciclo = u
                return True

        self.na_pilha[u] = False  # Desempilha após concluir
        return False
```

1. Empilhar e Desempilhar

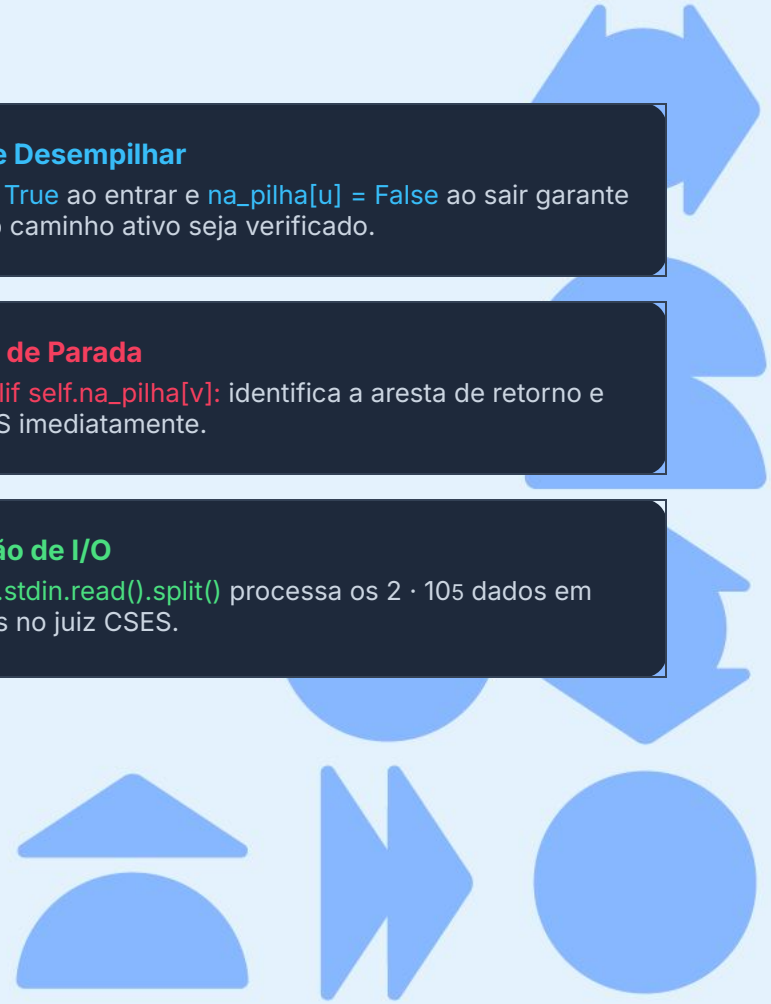
na_pilha[u] = True ao entrar e na_pilha[u] = False ao sair garante que apenas o caminho ativo seja verificado.

2. Condição de Parada

A instrução elif self.na_pilha[v]: identifica a aresta de retorno e encerra a DFS imediatamente.

3. Otimização de I/O

O uso de sys.stdin.read().split() processa os $2 \cdot 10^5$ dados em apenas 0.32 s no juiz CSES.



Justificativa das Complexidades de Tempo e Memória

COMPLEXIDADE DE TEMPO

O(N + M)

- **Vértices (N):** Cada nó entra e sai da pilha de recursão no máximo uma vez.
- **Arestas (M):** Cada arco é inspecionado exatamente uma vez na lista de adjacências.
- **Tempo Real CSES:** 0.32 s (Limite: 1.00 s).

⏱️ Tempo Otimizado

COMPLEXIDADE DE MEMÓRIA

O(N + M)

- **Grafo:** Lista de adjacências armazena N ponteiros e M arestas.
- **Auxiliares:** Vetores `na_pilha`, `visitado` e `pai` de tamanho N + 1.
- **Uso Real CSES:** 42.1 MB (Limite: 512 MB).

Baixo Consumo de Memória


Unifor